

Zotero–Obsidian Literature Management System

Technical Specification and Implementation Plan

Status: Proposed

Version: 0.1

Date: 5 August 2026

Primary interface: Obsidian

Supported agents: Claude Code and OpenAI Codex

Implementation language: Python 3.11+

1. Purpose

This project will provide a lightweight, local-first literature management system for academic research.

The system will connect Zotero with an Obsidian vault and allow the user to:

- Manage papers and PDFs in Zotero.
- Represent each paper as a structured Markdown file in Obsidian.
- Track personal reading status separately from AI review status.
- Ask Claude Code or Codex to read and summarize a paper.
- Preserve a clear distinction between human-authored and AI-authored notes.
- Generate concise AI reviews and reading recommendations.
- Reuse existing tags and propose new tags.
- Apply approved tags back to Zotero.
- Keep the entire knowledge layer inspectable, editable, and version-controllable.

The first version is deliberately limited to **literature management**. It will not yet implement a general research knowledge graph, embeddings, semantic search, project management, claim extraction, experiment tracking, or full RAG.

The architecture should nevertheless allow those components to be added later without restructuring the existing literature database.

2. Design principles

2.1 Zotero remains the bibliographic authority

Zotero owns:

- Bibliographic metadata.

- Publication identifiers.
- PDF attachments.
- Zotero annotations.
- Zotero collections.
- Existing Zotero tags.
- The immutable Zotero item key.

The system must not directly modify Zotero's internal SQLite database.

Communication with Zotero will occur through the Zotero local HTTP API. Current Zotero versions expose the API under `http://localhost:23119/api/`. The local API must be enabled in Zotero's advanced settings. It supports reading local library data, resolving attachment paths, and, in Zotero 10+, authorized write operations.

2.2 Markdown remains the research-workflow authority

Markdown files own:

- Personal reading status.
- Human-authored notes.
- AI review state.
- AI-generated summaries.
- AI reading recommendations.
- Curated literature tags.
- Proposed new tags.
- Whether an AI review has been human-verified.

All intellectually meaningful information added outside Zotero must remain visible as ordinary Markdown.

2.3 Human and AI activity must remain distinguishable

The system must never imply that the user has read a paper merely because an AI agent reviewed it.

Human and AI state must therefore use separate properties:

```
human_read_status: unread
ai_review_status: reviewed
```

AI-generated content must be placed in a clearly marked managed block and include:

- The agent used.
- The model, where available.
- The review date.
- Whether the full paper or only part of it was available.
- Whether the AI review has been checked by the user.

2.4 AI notes must be compact

The default AI review must be no longer than approximately 200 words, excluding metadata and tags.

It should focus on:

- What the paper contributes.
- Why it matters.
- Its main limitation.
- Whether the user should read it.
- Relevant tags.

The system should not generate long section-by-section summaries unless explicitly requested.

2.5 Generated state must be disposable

Extracted PDF text, logs, caches, and synchronization metadata must be stored under `.research/`.

These files must be rebuildable and should normally be excluded from Git.

2.6 Synchronization must be conservative

The system must:

- Never overwrite human notes.
- Never overwrite an AI review except through an explicit new AI review.
- Never delete Zotero tags automatically.
- Never delete a paper note merely because an item is missing from Zotero.
- Never modify bibliographic metadata through tag synchronization.
- Use version checks when writing to Zotero.

3. Scope

3.1 Included in version 0.1

Version 0.1 includes:

1. A structured Obsidian vault.
2. Zotero metadata synchronization.
3. Stable Better BibTeX citation keys.
4. One Markdown note per paper.
5. PDF attachment discovery.
6. PDF text extraction.
7. Human reading-status tracking.

8. AI review-status tracking.
9. Compact AI summaries.
10. AI reading recommendations.
11. Existing-tag selection.
12. New-tag suggestions.
13. Approved tag synchronization to Zotero.
14. Validation of Markdown files.
15. Support for Claude Code through `CLAUDE.md`.
16. Support for Codex through `AGENTS.md`.
17. Unit and integration tests.
18. Git-compatible operation.

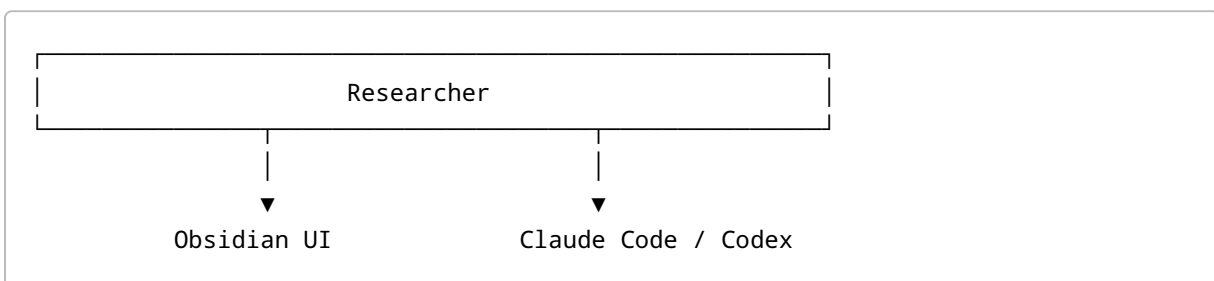
3.2 Explicitly excluded from version 0.1

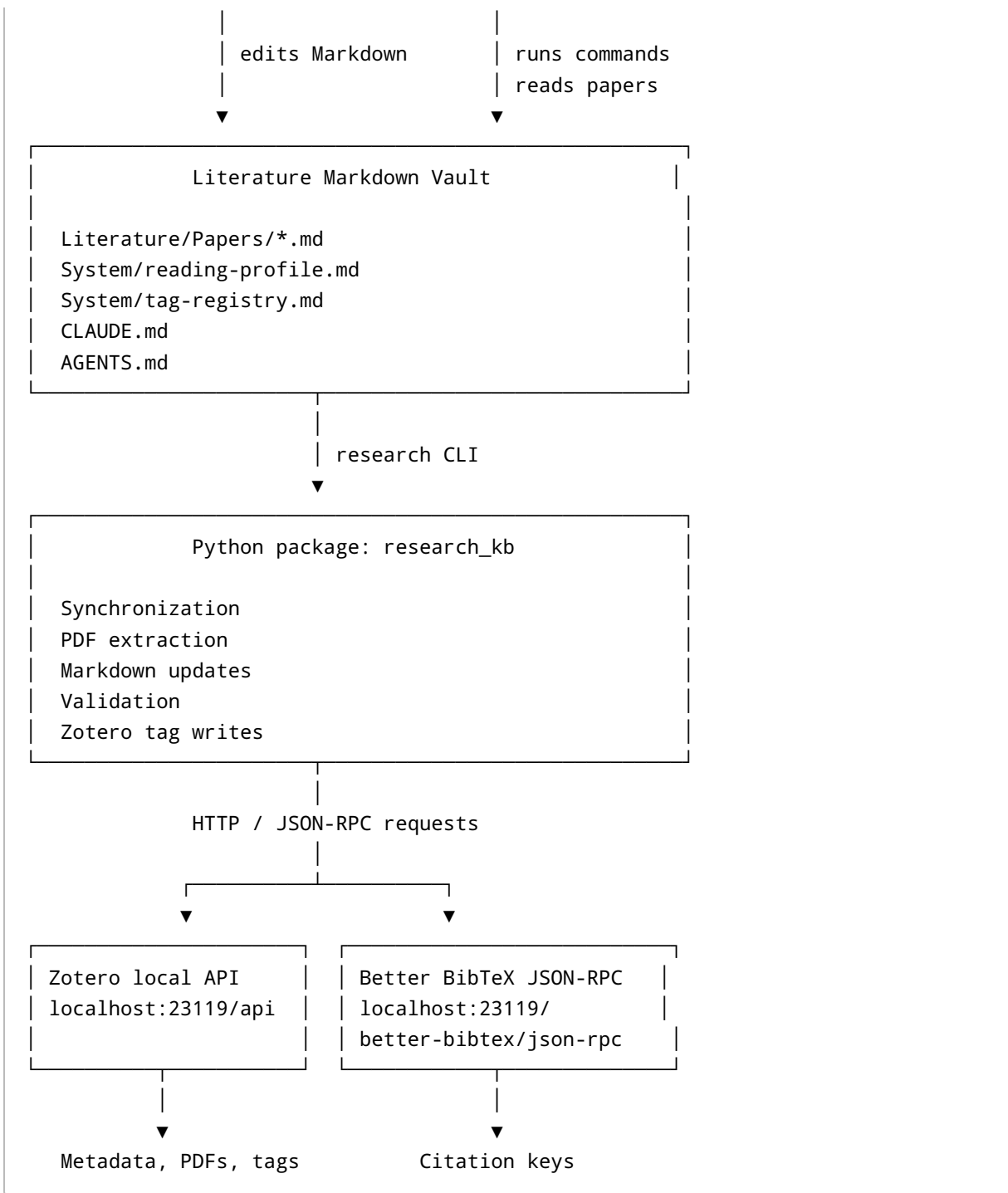
The following are postponed:

- Embedding generation.
- Vector databases.
- Semantic retrieval.
- SQLite full-text indexing.
- MCP servers.
- General research ideas.
- Concept notes.
- Claim and evidence graphs.
- Project notes.
- Experiment tracking.
- Manuscript generation.
- Automatic literature discovery.
- Automatic deletion or archiving.
- OCR for scanned PDFs.
- Automated AI invocation from Python.
- Continuous background synchronization.

Claude Code and Codex will be invoked interactively by the user. Python will provide deterministic data access and file-management commands, while the agents will perform the intellectual review.

4. System architecture





The Python package is the only component that needs to understand Zotero's API.

Claude Code and Codex interact with Zotero indirectly by running stable commands such as:

```
research sync
research extract chen2025flowdas
```

```
research validate
research push-tags chen2025flowdas
```

5. Repository and vault structure

The project root will also be the Obsidian vault.

```
research-literature/
├─ Literature/
│   ├── Papers/
│   │   ├── chen2025flowdas.md
│   │   └─ mucke2026energyconsistent.md
│   └─ Dashboards/
│       ├── Inbox.base
│       ├── Reading Queue.base
│       ├── AI Reviewed.base
│       ├── Human Read.base
│       └─ Recommended Reading.base
├─ System/
│   ├── Templates/
│   │   └─ Paper.md
│   ├── reading-profile.md
│   ├── tag-registry.md
│   ├── workflow.md
│   └─ schema.md
├─ src/
│   └─ research_kb/
│       ├── __init__.py
│       ├── cli.py
│       ├── config.py
│       ├── models.py
│       ├── zotero_client.py
│       ├── better_bibtex.py
│       ├── markdown_store.py
│       ├── sync_service.py
│       ├── extraction_service.py
│       ├── tag_service.py
│       ├── validation_service.py
│       └─ exceptions.py
```

```

├── logging_config.py
├── tests/
│   ├── unit/
│   │   ├── test_markdown_store.py
│   │   ├── test_tag_service.py
│   │   ├── test_models.py
│   │   └── test_validation_service.py
│   ├── integration/
│   │   ├── test_zotero_client.py
│   │   ├── test_better_bibtex.py
│   │   ├── test_sync_service.py
│   │   └── test_extraction_service.py
│   └── fixtures/
│       ├── zotero_item.json
│       ├── zotero_children.json
│       └── paper_note.md
├── .research/
│   ├── paper-text/
│   ├── sync-state.json
│   ├── credentials.json
│   ├── backups/
│   └── logs/
├── .github/
│   └── workflows/
│       └── test.yml
├── references.bib
├── CLAUDE.md
├── AGENTS.md
├── .env.example
├── .gitignore
├── pyproject.toml
├── README.md
└── SPEC.md

```

5.1 File naming

Paper notes will use the Better BibTeX citation key:

```
Literature/Papers/<citekey>.md
```

Example:

```
Literature/Papers/chen2025flowdas.md
```

The Zotero item key is the immutable internal identity.

The citation key is the human-readable external identity and filename.

```
zotero_key: ABCD1234  
citekey: chen2025flowdas
```

If a citation key changes, the software should rename the Markdown file while preserving its contents.

Citation keys should preferably be pinned in Better BibTeX when they are already used in notes, manuscripts, or external documents.

Better BibTeX supports stable citation-key generation, user-pinned keys, automatic exports, and direct citation-key lookup through its local JSON-RPC interface.

6. Component responsibilities

6.1 Zotero

Zotero stores:

- Journal articles, books, proceedings, and other references.
- Parent bibliographic items.
- PDF child attachments.
- Zotero annotations.
- Imported and manually added Zotero tags.
- Collections.
- Bibliographic metadata.

PDFs must not be duplicated into the Obsidian vault.

The Markdown note should contain Zotero links:

```
zotero_uri: zotero://select/library/items/ABCD1234
pdf_uri: zotero://open-pdf/library/items/PDFX5678
```

6.2 Better BibTeX

Better BibTeX provides:

- Stable citation keys.
- Citation-key lookup from Zotero item keys.
- Automatic export of `references.bib`.

The Python package will use Better BibTeX's JSON-RPC endpoint:

```
http://localhost:23119/better-bibtex/json-rpc
```

The `item.citationkey` method accepts Zotero item keys and returns their citation keys.

The bibliography export should be configured with “Keep updated”, causing Better BibTeX to refresh the export when items in the selected library or collection change.

6.3 Obsidian

Obsidian is the visual interface.

It will be used for:

- Browsing paper notes.
- Editing human notes.
- Filtering by properties.
- Viewing reading queues.
- Reviewing AI recommendations.
- Approving or rejecting proposed tags.
- Opening papers in Zotero.
- Comparing AI and human assessments.

No Zotero-specific Obsidian plugin is required for the core architecture.

Optional Obsidian plugins may be added later, but the project must remain functional without them.

6.4 Python CLI

The Python CLI handles:

- Zotero connectivity.

- Better BibTeX connectivity.
- Metadata import.
- Attachment lookup.
- PDF text extraction.
- Markdown creation and controlled updates.
- Tag reconciliation.
- Tag writes to Zotero.
- Validation.
- Diagnostics.

6.5 Claude Code and Codex

The agents handle:

- Reading extracted paper text.
- Summarizing the paper.
- Assessing relevance using the reading profile.
- Producing a recommendation.
- Selecting existing tags.
- Proposing new tags.
- Updating only AI-owned Markdown content.
- Running validation afterward.

Claude Code receives persistent project instructions from `CLAUDE.md`. Claude Code documents project-level `CLAUDE.md` files as persistent instructions loaded for sessions.

Codex receives repository instructions through `AGENTS.md`, as supported in the official Codex repository and documentation.

7. Data ownership model

Each field belongs to one of four owners.

7.1 Zotero-owned fields

These fields are synchronized from Zotero and must not be edited manually:

```
zotero_key:
zotero_version:
zotero_server_id:
title:
authors:
year:
publication:
```

```
volume:
issue:
pages:
doi:
url:
abstract:
zotero_collections:
zotero_tags:
zotero_uri:
pdf_attachment_key:
pdf_uri:
date_added:
date_modified:
```

7.2 Better-BibTeX-owned fields

```
citekey:
```

7.3 Human-owned fields

```
human_read_status:
human_read_date:
human_rating:
human_priority:
human_relevance:
```

The entire `## Human notes` section is also human-owned.

Agents must never change these fields or this section.

7.4 AI-owned fields

```
ai_review_status:
ai_review_scope:
ai_review_coverage:
ai_review_agent:
ai_review_model:
ai_review_date:
ai_review_version:
ai_review_human_verified:
ai_recommendation:
ai_recommendation_reason:
ai_relevance:
```

```
ai_recommendation_confidence:
ai_applied_tags:
ai_suggested_tags:
```

The managed `AI_REVIEW` Markdown block is AI-owned.

7.5 Shared curated fields

```
tags:
zotero_tag_sync:
zotero_tag_sync_date:
```

`tags` contains approved controlled-vocabulary tags.

An agent may add an existing canonical tag to `tags` when it has high confidence. New tags must first be placed in `ai_suggested_tags`.

8. Paper-note schema

A paper note will use the following structure.

```
---
schema_version: 1
type: paper

zotero_key: ABCD1234
zotero_version: 143
zotero_server_id: sPMHtLD6HHBd
citekey: chen2025flowdas

title: FlowDAS
authors:
  - First Author
  - Second Author
year: 2025
publication:
volume:
issue:
pages:
doi:
url:
abstract:
```

```
zotero_collections:
  - Data Assimilation
zotero_tags:
  - diffusion models
  - inverse problems

zotero_uri: zotero://select/library/items/ABCD1234
pdf_attachment_key: PDFX5678
pdf_uri: zotero://open-pdf/library/items/PDFX5678

date_added: 2026-08-05
date_modified: 2026-08-05

human_read_status: unread
human_read_date:
human_rating:
human_priority:
human_relevance:

ai_review_status: not-reviewed
ai_review_scope:
ai_review_coverage:
ai_review_agent:
ai_review_model:
ai_review_date:
ai_review_version: 0
ai_review_human_verified: false

ai_recommendation:
ai_recommendation_reason:
ai_relevance:
ai_recommendation_confidence:

tags: []
ai_applied_tags: []
ai_suggested_tags: []

zotero_tag_sync: not-synced
zotero_tag_sync_date:
---

# FlowDAS

## Why I saved this

## Human notes
```

```
### Summary

### Important results

### Critique

### Connections

## AI review

<!-- BEGIN MANAGED:AI_REVIEW -->

No AI review has been generated.

<!-- END MANAGED:AI_REVIEW -->

## Zotero annotations

<!-- BEGIN MANAGED:ZOTERO_ANNOTATIONS -->

No Zotero annotations have been imported.

<!-- END MANAGED:ZOTERO_ANNOTATIONS -->
```

9. Status definitions

9.1 Human reading status

Allowed values:

```
unread
queued
skimming
reading
read
```

Only the user may change this field.

Meaning:

- `unread` : The paper has not been read.
- `queued` : The user intends to read it.
- `skimming` : The user has performed or is performing a light review.

- `reading` : The user is actively reading it.
- `read` : The user considers the paper read.

9.2 AI review status

Allowed values:

```
not-reviewed
queued
reviewed
failed
outdated
```

Meaning:

- `not-reviewed` : No AI review exists.
- `queued` : The paper has been selected for AI review.
- `reviewed` : A valid AI review exists.
- `failed` : Review could not be completed.
- `outdated` : The source PDF or important Zotero metadata changed after the review.

9.3 AI review scope

Allowed values:

```
metadata-only
abstract-only
partial-text
full-text
```

9.4 AI review coverage

Allowed values:

```
complete
partial
unknown
```

Scope describes what source was intended to be reviewed.

Coverage describes whether that source was successfully available.

Examples:

```
ai_review_scope: full-text  
ai_review_coverage: complete
```

```
ai_review_scope: full-text  
ai_review_coverage: partial
```

The second case may occur when PDF extraction is incomplete, pages contain unreadable text, or important content is embedded mainly in figures.

9.5 AI reading recommendation

Allowed values:

```
must-read  
read  
skim  
skip  
uncertain
```

Interpretation:

- **must-read** : Central to the user's active research interests.
- **read** : Substantively relevant and worth reading fully.
- **skim** : Contains useful ideas or references but limited direct value.
- **skip** : Low expected value relative to the user's current interests.
- **uncertain** : The available evidence or extraction quality is insufficient.

9.6 Recommendation confidence

Allowed values:

```
low  
medium  
high
```

10. AI review contract

The AI review is a constrained output, not a general summary.

10.1 Required content

The review must include:

1. Up to three summary bullets.
2. A one- or two-sentence statement of the main contribution.
3. One or two important limitations.
4. A reading recommendation.
5. A concise reason for the recommendation.
6. Existing tags selected from the registry.
7. At most two proposed new tags.

10.2 Length

The prose portion should remain below approximately 200 words.

The default target is 120–180 words.

10.3 Required managed-block format

```
<!-- BEGIN MANAGED:AI_REVIEW -->

> **Generated by:** Claude Code
> **Model:** Claude model identifier, if available
> **Date:** 2026-08-05
> **Scope:** Full text
> **Coverage:** Complete
> **Human verified:** No

### Summary

- The paper introduces ...
- It evaluates ...
- The main result is ...

### Main contribution

One or two concise sentences.

### Main limitations
```

```

- Limitation one.
- Limitation two.

### Reading recommendation

**Read.** The method is directly relevant to posterior sampling for physical
inverse problems, and the full paper contains implementation details not
recoverable from the abstract.

### Tags

**Applied:** `method/diffusion-models`, `task/data-assimilation`
**Suggested:** `method/likelihood-guided-sampling`

<!-- END MANAGED:AI_REVIEW -->

```

10.4 Prohibited behavior

The agent must not:

- Change `human_read_status`.
- Change `human_read_date`.
- Change `human_rating`.
- Write under `## Human notes`.
- Claim full-paper review when only the abstract was available.
- Add more than five existing tags automatically.
- add more than two proposed new tags.
- Push tags to Zotero unless explicitly instructed.
- Treat its own summary as verified human knowledge.
- Invent page references.
- hide extraction limitations.
- Mark `ai_review_human_verified: true`.

Only the user may mark an AI review as verified.

11. Reading profile

`System/reading-profile.md` defines the criteria used for recommendations.

Initial content:

```

# Reading profile

## Primary interests

```

- Generative modelling for physical systems
- Bayesian inverse problems
- Data assimilation
- Scientific machine learning
- PDE surrogate modelling
- Physics-constrained learning
- Structure-preserving learning
- Scientific and physical foundation models
- Sim-to-real transfer for physical systems

Especially valuable papers

A paper is especially valuable when it:

- Introduces reusable methodology rather than only an isolated application.
- Addresses sparse, noisy, or indirect observations.
- Contains careful comparisons, ablations, or uncertainty analysis.
- Studies physical consistency or preservation of invariants.
- Discusses generalization, adaptation, transfer, or sim-to-real behaviour.
- Provides open-source code or sufficiently detailed algorithms.
- Connects generative modelling with inverse problems or data assimilation.

Lower-priority papers

A paper is generally lower priority when it:

- Reports only a small benchmark improvement.
- Applies a standard model without methodological insight.
- Focuses on a domain with little transfer to physical systems.
- Depends primarily on scale without analysing why the method works.
- Provides conclusions unsupported by suitable baselines or ablations.

Recommendation policy

Use ``must-read`` sparingly.

Prefer ``uncertain`` over an overconfident recommendation when:

- Only an abstract was available.
- PDF extraction was poor.
- Essential evidence was contained mainly in figures.
- The paper's relevance depends on details not available in the extracted text.

12. Tag system

12.1 Canonical namespaces

Initial namespaces:

```
domain/  
method/  
task/  
property/  
model/  
data/
```

Examples:

```
domain/fluid-dynamics  
domain/weather  
domain/subsurface-flow  
  
method/diffusion-models  
method/flow-matching  
method/stochastic-interpolants  
method/neural-operators  
method/ensemble-kalman-methods  
  
task/data-assimilation  
task/inverse-problems  
task/surrogate-modelling  
task/super-resolution  
  
property/physics-constrained  
property/probabilistic  
property/structure-preserving  
property/geometry-aware  
  
model/foundation-model  
model/transformer  
model/unet  
  
data/sparse-observations  
data/simulation-data  
data/remote-sensing
```

12.2 Tag registry

`System/tag-registry.md` is the controlled vocabulary.

Each tag should include a brief definition:

```
# Tag registry

## Domain

### `domain/fluid-dynamics`

Papers concerning fluid flow, CFD, Navier-Stokes equations, turbulence, LES, or
related flow systems.

### `domain/weather`

Papers concerning weather prediction, atmospheric modelling, meteorology, or
numerical weather models.

## Method

### `method/diffusion-models`

Score-based or diffusion-based generative modelling.

### `method/stochastic-interpolants`

Methods explicitly using stochastic interpolants or closely equivalent
formulations.
```

12.3 Agent tag rules

The agent must:

1. Read `System/tag-registry.md`.
2. Prefer an existing tag over creating a synonym.
3. Apply no more than five existing tags.
4. Suggest no more than two new tags.
5. Use lowercase kebab-case.
6. Use an approved namespace.
7. Avoid tags that describe incidental implementation details.
8. Avoid author names, venue names, and years as tags.
9. Avoid duplicating information already represented by bibliographic metadata.

12.4 Tag states

Approved tags:

```
tags:
  - method/diffusion-models
  - task/data-assimilation
```

Tags selected by the agent from the existing registry:

```
ai_applied_tags:
  - method/diffusion-models
  - task/data-assimilation
```

Unapproved proposals:

```
ai_suggested_tags:
  - method/likelihood-guided-sampling
```

A suggested tag must not be written to Zotero until the user approves it by:

1. Adding it to `System/tag-registry.md`.
2. Moving it into `tags`.
3. Removing it from `ai_suggested_tags`.

13. Zotero communication specification

13.1 Local API discovery

The Python client must first call:

```
GET http://localhost:23119/api/
```

It must read:

```
Zotero-API-Version
Zotero-Server-ID
Zotero-Schema-Version
```

The client must fail with a clear message if:

- Zotero is not running.
- The API is disabled.
- The port is unavailable.
- The API version is unsupported.

Zotero documents that the local API must be explicitly enabled and that requests are served from the local database.

13.2 Server identity

The cached sync state must be partitioned by `Zotero-Server-ID`.

If the server ID changes:

- Existing version cursors must be invalidated.
- A full resynchronization must be performed.
- No cached object version from the previous server may be reused.

Zotero 10+ treats local object versions as specific to the local Zotero instance, and official guidance requires clients to reset cached state when the server ID changes.

13.3 Reading items

The client retrieves top-level bibliographic items through the local API.

It must ignore:

- Child PDF attachment objects during the main paper-note loop.
- Standalone notes unless annotation import is implemented.
- Feed items.
- Trashed items by default.
- Unsupported item types unless explicitly configured.

Supported initial types should include:

```
journalArticle
conferencePaper
preprint
book
bookSection
thesis
report
document
```

13.4 Incremental synchronization

The client stores the latest local library version in:

```
.research/sync-state.json
```

Subsequent synchronizations use Zotero's `since` filtering to retrieve changed objects.

If incremental synchronization fails or the server ID changes, the client performs a full sync.

13.5 PDF attachment resolution

For a bibliographic item:

1. Retrieve child items.
2. Select attachment children with MIME type `application/pdf`.
3. Prefer a non-deleted locally available PDF.
4. Store the chosen attachment key.
5. Resolve its local file URL through:

```
GET /api/users/0/items/<attachment-key>/file/view/url
```

The Zotero local API explicitly supports returning local attachment URLs through its file endpoints.

If several PDFs exist, selection order should be:

1. Primary attached PDF.
2. Most recently modified PDF.
3. First available PDF.

A future version may allow explicit attachment selection.

13.6 Local write authorization

For Zotero 10+, the tool should use the local API for writes.

Authorization flow:

1. Read `Zotero-Server-ID`.
2. Send:

```
POST /api/local/authorize
Content-Type: application/json
```

```
Zotero-Server-ID: <server-id>
```

```
{  
  "appName": "Research Literature Manager"  
}
```

1. Ask the user to approve the Zotero dialog.
2. Store remembered keys in:

```
.research/credentials.json
```

1. Ensure this file is excluded from Git.
2. Retry authorization after `401 Unauthorized`.

Zotero local write keys are granted through a user confirmation dialog and are distinct from zotero.org API keys.

13.7 Web API fallback

The implementation may optionally support the Zotero Web API when:

- The installed Zotero version does not support local writes.
- Local write authorization is unavailable.
- The user explicitly selects Web API mode.

Web API credentials must be stored in environment variables or an ignored local credentials file.

The initial implementation should prioritize local writes.

13.8 Tag write operation

The tag push algorithm must:

1. Retrieve the latest item.
2. Read the item's current version.
3. Read all current Zotero tags.
4. Read approved Markdown `tags`.
5. Restrict outgoing tags to configured namespaces.
6. Merge approved tags with existing Zotero tags.
7. Never delete existing tags.
8. Skip the write if no new tags exist.
9. Send a partial `PATCH` containing only the complete merged `tags` array.
10. Include the current item version.
11. Retry once after a `412 Precondition Failed`.
12. Report unresolved conflicts.

Zotero's `PATCH` semantics preserve omitted properties, but array properties included in a patch are treated as complete arrays. Therefore the client must send the existing and new tags together. Zotero also requires current version information for `PATCH` writes.

14. Better BibTeX integration

14.1 Readiness check

The client calls the Better BibTeX JSON-RPC method:

```
api.ready
```

If unavailable, `research doctor` should report:

```
Better BibTeX is unavailable.  
Install or enable Better BibTeX and restart Zotero.
```

14.2 Citation-key lookup

For each Zotero item key, call:

```
item.citationkey
```

Input:

```
["0:ABCD1234"]
```

Output:

```
{  
  "0:ABCD1234": "chen2025flowdas"  
}
```

14.3 Bibliography export

The user should configure an automatic Better BibTeX export to:

```
references.bib
```

The Python package does not need to regenerate this file on every sync.

`research doctor` should warn if it is absent.

15. PDF extraction specification

15.1 Extraction library

Use PyMuPDF as the initial PDF extraction library.

15.2 Output format

Extracted text is stored as:

```
.research/paper-text/<citekey>.md
```

Example:

```
---
citekey: chen2025flowdas
zotero_key: ABCD1234
attachment_key: PDFX5678
source_mtime: 1785924000
source_size: 2459321
extracted_at: 2026-08-05T12:00:00+02:00
extractor: pymupdf
extractor_version: 1.26.0
pages: 24
---

# Extracted paper: chen2025flowdas

<!-- PAGE 1 -->

Page-one text.

<!-- PAGE 2 -->
```

Page-two text.

Page markers allow agents to associate claims with page numbers.

15.3 Cache validity

The extraction cache is valid when:

- The attachment key is unchanged.
- The file modification time is unchanged.
- The file size is unchanged.
- The extractor version is unchanged.

Otherwise, extraction must run again.

15.4 Extraction diagnostics

The extractor should calculate:

- Number of pages.
- Number of characters per page.
- Number of empty pages.
- Total extracted characters.
- Fraction of pages with very little text.

A warning should be emitted when:

- More than 20% of pages are empty.
- Average text per page is suspiciously low.
- The PDF appears scanned.
- Extraction fails on one or more pages.
- The file is encrypted.

The system will not use OCR in version 0.1.

When extraction is poor, the AI review must use:

```
ai_review_coverage: partial
```

or:

```
ai_review_status: failed
```

16. CLI specification

The executable name is:

```
research
```

It is installed through `pyproject.toml`:

```
[project.scripts]
research = "research_kb.cli:app"
```

16.1 `research doctor`

Purpose:

- Test Zotero connectivity.
- Test local API status.
- Display Zotero API version.
- Display Zotero server ID.
- Test Better BibTeX.
- Check `references.bib`.
- Check vault paths.
- Test write authorization status.
- Check required configuration.

Example:

```
research doctor
```

16.2 `research sync`

Purpose:

- Import changed Zotero items.
- Resolve citation keys.
- Create paper notes.
- Update Zotero-owned metadata.
- Update attachment references.
- Mark outdated AI reviews.
- Update sync state.

Options:

```
research sync
research sync --dry-run
research sync --full
research sync --item ABCD1234
research sync --citekey chen2025flowdas
```

16.3 research extract

Purpose:

- Locate a paper's PDF.
- Extract page-aware text.
- Cache the output.
- Report extraction quality.

Examples:

```
research extract chen2025flowdas
research extract chen2025flowdas --force
```

16.4 research review-context

Purpose:

Prepare and print the paths an agent should read.

```
research review-context chen2025flowdas
```

Expected output:

```
Paper note:
Literature/Papers/chen2025flowdas.md

Extracted paper:
.research/paper-text/chen2025flowdas.md

Reading profile:
System/reading-profile.md

Tag registry:
System/tag-registry.md
```

This command may run extraction automatically when required.

16.5 `research validate`

Purpose:

Validate the entire vault or one note.

Examples:

```
research validate
research validate chen2025flowdas
```

The command must exit with a non-zero status when validation fails.

16.6 `research tags`

Purpose:

Display tag state.

```
research tags chen2025flowdas
```

Output should distinguish:

- Existing Zotero tags.
- Approved curated tags.
- AI-applied tags.
- Suggested new tags.
- Tags pending synchronization.

16.7 `research push-tags`

Purpose:

Apply approved namespaced tags to Zotero.

Examples:

```
research push-tags chen2025flowdas --dry-run
research push-tags chen2025flowdas
research push-tags --all --dry-run
```

Default behavior is add-only.

16.8 research authorize

Purpose:

Request or renew Zotero local write authorization.

```
research authorize
```

16.9 research show

Purpose:

Display a concise status summary.

```
research show chen2025flowdas
```

Suggested output:

```
Title: FlowDAS
Human status: unread
AI status: reviewed
AI scope: full-text
Recommendation: read
Tags synchronized: yes
PDF available: yes
Extraction: complete
```

17. Synchronization algorithm

17.1 New item

When a new Zotero item is found:

1. Validate that it is a supported bibliographic type.
2. Fetch its Better BibTeX citation key.
3. Fetch child attachments.
4. Select a PDF.
5. Build Zotero and PDF URIs.
6. Render the paper template.
7. Set:

```
human_read_status: unread
ai_review_status: not-reviewed
ai_review_version: 0
```

1. Write the note atomically.
2. Record the Zotero version.

17.2 Existing item

When a matching note exists:

1. Parse the frontmatter.
2. Match using `zotero_key`.
3. Update only Zotero-owned fields.
4. Preserve human-owned fields.
5. Preserve AI-owned fields.
6. Preserve all unmanaged Markdown sections.
7. Update Zotero annotations inside their managed block.
8. Detect PDF changes.
9. Mark the AI review as outdated when appropriate.
10. Write atomically only if content changed.

17.3 Matching priority

Use:

1. Exact `zotero_key`.
2. Exact citation key.
3. Manual conflict report.

Do not use fuzzy title matching automatically.

17.4 Citation-key changes

When the Zotero key matches but the citation key changes:

1. Update `citekey`.
2. Rename the Markdown file.
3. Rename cached extraction files.
4. Preserve Git history where possible.
5. Warn that external links may need updating.
6. Do not automatically rewrite arbitrary files in version 0.1.

17.5 Missing Zotero items

If a Markdown note refers to an item no longer returned by Zotero:

```
zotero_missing: true
```

The note must not be deleted or moved automatically.

17.6 Outdated AI reviews

Set:

```
ai_review_status: outdated
```

when:

- The PDF attachment key changes.
- The PDF modification time changes.
- The PDF file size changes materially.
- The title or abstract changes after an abstract-only review.
- The user explicitly requests invalidation.

Minor tag or collection changes should not invalidate the review.

18. Markdown-update safety

18.1 Managed blocks

Only explicitly managed blocks may be replaced programmatically.

Required markers:

```
<!-- BEGIN MANAGED:AI_REVIEW -->
<!-- END MANAGED:AI_REVIEW -->
```

```
<!-- BEGIN MANAGED:ZOTERO_ANNOTATIONS -->
<!-- END MANAGED:ZOTERO_ANNOTATIONS -->
```

18.2 Atomic writes

All Markdown updates must:

1. Render content in memory.
2. Write to a temporary file in the same directory.
3. Flush and close the file.
4. Replace the original atomically.
5. Retain a backup when parsing or validation detects risk.

18.3 Parse failures

If frontmatter or managed markers are malformed:

- Do not rewrite the file.
- Report the exact file and error.
- Save no partial output.
- Exit non-zero.

18.4 Backups

Before a potentially destructive schema migration, copy affected files to:

```
.research/backups/<timestamp>/
```

Routine metadata updates do not require permanent backups when Git is used.

19. Validation rules

`research validate` must check:

19.1 Required identity

Every paper must contain:

```
type
schema_version
zotero_key
citekey
title
```

19.2 Valid enums

All status and recommendation fields must use allowed values.

19.3 Human/AI separation

Validation fails if:

- AI-owned automation changed human fields during an agent workflow.
- An AI review exists without an agent identifier.
- A full-text review exists without a corresponding extraction cache.
- `ai_review_human_verified` is not Boolean.
- `human_read_status: read` is inferred merely from AI state.

19.4 Tag validation

Validation fails when:

- A curated tag has no namespace.
- A tag contains spaces or uppercase characters.
- An AI review applies more than five existing tags.
- An AI review suggests more than two new tags.
- A suggested tag is also treated as synchronized without approval.
- A tag uses an unknown namespace.

Unknown tags in `tags` may produce either an error or warning, configurable through settings.

19.5 Managed-block integrity

Check:

- Each opening marker has exactly one closing marker.
- Blocks do not overlap.
- Required blocks exist.
- No duplicate managed block exists.

19.6 Recommendation completeness

When:

```
ai_review_status: reviewed
```

the following must be present:

```
ai_review_scope  
ai_review_coverage  
ai_review_agent  
ai_review_date  
ai_recommendation  
ai_recommendation_reason  
ai_recommendation_confidence
```

20. Agent instructions

`CLAUDE.md` and `AGENTS.md` should contain equivalent literature-review instructions.

Suggested shared content:

```
# Literature management instructions  
  
This repository is an Obsidian literature vault connected to Zotero.  
  
## Core rules  
  
- Zotero owns bibliographic metadata and PDFs.  
- Human notes are owned exclusively by the user.  
- Never change `human_read_status`, `human_read_date`,  
  `human_rating`, `human_priority`, or `human_relevance`.  
- Never write inside `## Human notes`.  
- Update only AI-owned frontmatter fields and the  
  `MANAGED:AI_REVIEW` block.  
- Never claim to have read the full paper unless the full extracted  
  text is available.  
- Record the actual review scope and extraction coverage.  
- Keep AI reviews below 200 words unless explicitly requested otherwise.  
- Use at most three summary bullets.  
- Include the main contribution, the most important limitations,  
  and a reading recommendation.  
- Base recommendations on `System/reading-profile.md`.  
- Read `System/tag-registry.md` before assigning tags.  
- Prefer existing tags.  
- Apply at most five existing tags.
```

- Suggest at most two new tags.
- Put proposed new tags in ``ai_suggested_tags``.
- Do not approve new tags automatically.
- Do not push tags to Zotero unless explicitly asked.
- Run ``research validate <citekey>`` after editing a paper note.
- If extraction is incomplete, say so explicitly.
- Do not treat an abstract as evidence for details found only in the body.

21. User workflows

21.1 Add a new paper

1. Save the paper through Zotero Connector.
2. Confirm that Zotero has metadata and a PDF.
3. Better BibTeX generates the citation key.
4. Run:

```
research sync
```

1. Open the new note in Obsidian.
2. Optionally set:

```
human_read_status: queued
human_priority: 4
human_relevance: 5
```

21.2 Ask Claude Code to review a paper

Example:

```
Review chen2025flowdas.
```

```
Use the full paper if available. Keep the review concise and update only
the AI-owned fields and AI review block. Use my reading profile, prefer
existing tags, and suggest at most two new tags. Do not change my human
reading status or push anything to Zotero.
```

Claude Code should:

```
research review-context chen2025flowdas
```

Then read:

```
Literature/Papers/chen2025flowdas.md  
.research/paper-text/chen2025flowdas.md  
System/reading-profile.md  
System/tag-registry.md
```

After editing:

```
research validate chen2025flowdas
```

21.3 Ask Codex to review a paper

Use the same request.

Codex follows `AGENTS.md`, runs the same CLI commands, reads the same files, and produces the same schema.

The result should not depend on which agent was used, except for:

```
ai_review_agent: codex  
ai_review_model: <model identifier>
```

21.4 Approve a proposed tag

Suppose the AI proposes:

```
ai_suggested_tags:  
- method/likelihood-guided-sampling
```

The user:

1. Checks that no equivalent tag exists.
2. Adds the tag and definition to `System/tag-registry.md`.
3. Moves the tag into `tags`.
4. Removes it from `ai_suggested_tags`.
5. Runs:

```
research validate chen2025flowdas
research push-tags chen2025flowdas --dry-run
research push-tags chen2025flowdas
```

21.5 Read the paper personally

After reading:

```
human_read_status: read
human_read_date: 2026-08-07
human_rating: 4
```

The user writes under:

```
## Human notes
```

The AI review remains unchanged.

This makes it possible to compare:

- The AI's initial recommendation.
- The user's eventual judgement.
- Differences between AI and human summaries.

21.6 Re-review an updated preprint

After Zotero receives a new PDF version:

```
research sync
```

The system detects the changed attachment and sets:

```
ai_review_status: outdated
```

The user asks an agent to review the paper again.

The agent increments:

```
ai_review_version: 2
```

Git retains the old review in history.

22. Obsidian dashboards

Dashboards should be implemented with Obsidian Bases where practical.

22.1 Inbox

Filter:

```
type == "paper"
human_read_status == "unread"
```

Columns:

```
title
authors
year
human_priority
human_relevance
ai_review_status
ai_recommendation
date_added
```

22.2 Reading queue

Filter:

```
human_read_status == "queued"
```

Sort by:

1. `human_priority`, descending.
2. `human_relevance`, descending.
3. `date_added`, ascending.

22.3 AI reviewed

Filter:

```
ai_review_status == "reviewed"
```

Columns:

```
title  
ai_recommendation  
ai_relevance  
ai_recommendation_confidence  
ai_review_scope  
ai_review_agent  
ai_review_date
```

22.4 Recommended reading

Filter:

```
ai_recommendation in ["must-read", "read"]  
human_read_status in ["unread", "queued"]
```

22.5 Human read

Filter:

```
human_read_status == "read"
```

22.6 Read but AI-unverified

Filter:

```
ai_review_status == "reviewed"  
ai_review_human_verified == false  
human_read_status == "read"
```

This view helps the user verify or correct the AI review after reading the paper.

23. Configuration

Use `pyproject.toml` for package configuration and `.env` for local secrets or overrides.

23.1 Example `.env`

```
RESEARCH_VAULT_PATH=.
ZOTERO_LOCAL_API=http://localhost:23119/api
BETTER_BIBTEX_RPC=http://localhost:23119/better-bibtex/json-rpc
ZOTERO_LIBRARY_TYPE=user
ZOTERO_LIBRARY_ID=0
RESEARCH_LOG_LEVEL=INFO
```

Optional Web API fallback:

```
ZOTERO_WEB_API_KEY=
ZOTERO_WEB_LIBRARY_ID=
```

23.2 Application configuration

A checked-in configuration file may be added later:

```
research.toml
```

Potential contents:

```
[vault]
papers_dir = "Literature/Papers"
templates_dir = "System/Templates"
tag_registry = "System/tag-registry.md"
reading_profile = "System/reading-profile.md"

[zotero]
local_api = "http://localhost:23119/api"
library_type = "user"
library_id = 0
write_mode = "local"

[review]
max_words = 200
max_applied_tags = 5
max_suggested_tags = 2

[tags]
allowed_namespaces = [
    "domain",
```

```
"method",  
"task",  
"property",  
"model",  
"data",  
]  
add_only = true
```

24. Python package design

24.1 cli.py

Responsibilities:

- Define Typer commands.
- Convert domain exceptions into readable CLI errors.
- Configure logging.
- Return meaningful exit codes.

24.2 config.py

Responsibilities:

- Load environment variables.
- Resolve vault-relative paths.
- Validate configuration.
- Expose a typed settings object.

Use Pydantic Settings or an equivalent typed configuration layer.

24.3 models.py

Define typed models:

```
class ZoteroItem  
class ZoteroAttachment  
class PaperNote  
class AIReviewMetadata  
class SyncState  
class ExtractionMetadata  
class TagRegistry
```

All external JSON and frontmatter must be validated before use.

24.4 `zotero_client.py`

Responsibilities:

- Discover the local API.
- Read server identity.
- Retrieve items.
- Retrieve changed items.
- Retrieve child attachments.
- Resolve attachment paths.
- Authorize writes.
- Patch tags.
- Handle version conflicts.
- Optionally support the Web API fallback.

24.5 `better_bibtex.py`

Responsibilities:

- Check readiness.
- Resolve citation keys.
- Report missing keys.
- Optionally trigger or verify auto-export.

24.6 `markdown_store.py`

Responsibilities:

- Parse YAML frontmatter.
- Preserve field ordering where practical.
- Update only owned fields.
- Find and replace managed blocks.
- Render new paper notes.
- Perform atomic writes.
- Rename notes when citekeys change.

24.7 `sync_service.py`

Responsibilities:

- Coordinate Zotero, Better BibTeX, and Markdown.
- Apply synchronization rules.
- Detect missing items.
- Detect changed PDFs.

- Mark AI reviews outdated.
- Persist sync state.

24.8 `extraction_service.py`

Responsibilities:

- Resolve the PDF path.
- Validate cache state.
- Extract text page by page.
- Write page-aware Markdown.
- Compute extraction-quality metrics.
- Report partial extraction.

24.9 `tag_service.py`

Responsibilities:

- Parse the tag registry.
- Validate names and namespaces.
- Calculate approved outgoing tags.
- Compare Markdown tags with Zotero tags.
- Generate dry-run reports.
- Perform add-only tag writes.

24.10 `validation_service.py`

Responsibilities:

- Validate schemas.
- Validate enum values.
- Validate managed blocks.
- Validate human/AI ownership.
- Validate tag limits.
- Validate review completeness.
- Validate extraction/review consistency.

24.11 `exceptions.py`

Define domain-specific errors such as:

```
class ZoteroUnavailableError
class ZoteroAuthorizationError
class ZoteroConflictError
class BetterBibTeXUnavailableError
class CitationKeyMissingError
```

```
class PDFNotFoundError
class PDFExtractionError
class MarkdownParseError
class ManagedBlockError
class ValidationError
```

25. Dependencies

Initial production dependencies:

```
dependencies = [
    "httpx>=0.28",
    "pydantic>=2.0",
    "pydantic-settings>=2.0",
    "python-frontmatter>=1.1",
    "PyYAML>=6.0",
    "PyMuPDF>=1.26",
    "typer>=0.16",
    "rich>=14.0",
]
```

Development dependencies:

```
[dependency-groups]
dev = [
    "pytest>=8.0",
    "pytest-cov>=6.0",
    "respx>=0.22",
    "ruff>=0.12",
    "mypy>=1.17",
]
```

Use dependency ranges rather than hard-pinning every transitive package.

A lock file should be committed if the project uses `uv`.

26. Security and privacy

26.1 Local API exposure

The Zotero local API must remain bound to localhost.

The port must not be forwarded or exposed to other machines. Zotero notes that local read access is unauthenticated and therefore should not be externally exposed.

26.2 Credentials

The following must never be committed:

```
.env
.research/credentials.json
Zotero API keys
local write keys
```

26.3 Agent access

Claude Code and Codex will have access to:

- The Markdown vault.
- Extracted paper text.
- The local project files.
- Shell commands permitted by the user.

They should not receive direct credentials unless absolutely required.

The agents should invoke `research push-tags`; they should not construct raw Zotero API requests themselves.

26.4 Destructive actions

Version 0.1 must not expose commands for:

- Deleting Zotero items.
 - Deleting Zotero tags.
 - Deleting attachments.
 - Replacing PDFs.
 - Modifying bibliographic metadata.
-

27. Logging

Logs are stored under:

```
.research/logs/
```

Each command should log:

- Timestamp.
- Command name.
- Zotero server ID.
- Item keys affected.
- Files created or modified.
- Tags added.
- Errors and retries.

Do not log:

- API keys.
- Local authorization keys.
- Complete paper text.
- Sensitive environment variables.

Console output should remain concise. Detailed diagnostics belong in log files or behind `--verbose`.

28. Testing strategy

28.1 Unit tests

Test:

- Tag normalization.
- Tag namespace validation.
- Managed-block replacement.
- Frontmatter updates.
- Ownership preservation.
- Status validation.
- Recommendation validation.
- Cache-validity checks.
- Citation-key filename handling.

28.2 Mocked API tests

Use `respx` or an equivalent HTTP mock to test:

- Zotero discovery.
- Item retrieval.
- Attachment retrieval.
- Write authorization.
- Successful tag patch.
- `401` reauthorization.
- `412` version conflicts.
- Unavailable Zotero.
- Unavailable Better BibTeX.

28.3 Integration tests

With an isolated Zotero test profile:

1. Create a test item.
2. Attach a PDF.
3. Run sync.
4. Verify note creation.
5. Extract the PDF.
6. Add approved tags.
7. Push tags.
8. Verify immediate Zotero update.
9. Modify the PDF.
10. Verify the AI review becomes outdated.

28.4 Regression tests

Fixtures should cover:

- Existing human notes.
- Existing AI review.
- Missing managed markers.
- Unicode author names.
- Multiple PDF attachments.
- Citation-key changes.
- Empty abstracts.
- Items without PDFs.
- Malformed frontmatter.
- Existing non-namespaced Zotero tags.

28.5 Acceptance test

A full manual acceptance test is successful when:

1. A paper is added to Zotero.
 2. `research sync` creates a valid Markdown note.
 3. `research extract` creates readable page-aware text.
 4. Claude Code reviews the paper without touching human fields.
 5. Codex can perform the same workflow.
 6. Validation passes.
 7. An approved tag is pushed to Zotero.
 8. Existing Zotero tags remain intact.
 9. Obsidian displays the note and properties correctly.
 10. A changed PDF marks the review outdated.
-

29. Implementation plan

Phase 0: Repository foundation

Deliverables:

- Repository structure.
- `pyproject.toml`.
- Package skeleton.
- Typer CLI.
- Configuration loader.
- Logging.
- `.gitignore`.
- Test framework.
- CI workflow.

Commands available:

```
research --help
research doctor
```

Completion criteria:

- Package installs with `uv sync` or `pip install -e .`.
- Tests run in CI.
- CLI starts successfully.

Phase 1: Zotero and Better BibTeX connectivity

Deliverables:

- Zotero local API discovery.
- API enablement diagnostics.
- Server ID handling.
- Item retrieval.
- Child attachment retrieval.
- Better BibTeX readiness check.
- Citation-key lookup.

Commands:

```
research doctor
research show --zotero-key ABCD1234
```

Completion criteria:

- A Zotero item can be retrieved.
- Its citation key can be resolved.
- Its PDF attachment can be identified.
- Failures produce actionable messages.

Phase 2: Markdown synchronization

Deliverables:

- Paper schema.
- Paper template.
- Frontmatter parsing.
- New-note creation.
- Controlled metadata updates.
- Managed blocks.
- Atomic writes.
- Sync-state persistence.
- Incremental sync.

Commands:

```
research sync
research sync --dry-run
research validate
```

Completion criteria:

- New Zotero items create notes.
- Existing human content survives repeated synchronization.
- Incremental synchronization works.
- Citation keys and filenames are consistent.

Phase 3: PDF extraction

Deliverables:

- PDF path resolution.
- PyMuPDF extraction.
- Page markers.
- Cache metadata.
- Extraction-quality reporting.
- Review-context command.

Commands:

```
research extract <citekey>
research review-context <citekey>
```

Completion criteria:

- A normal academic PDF is extracted successfully.
- Repeated extraction uses the cache.
- Changed PDFs invalidate the cache.
- Extraction problems are reported.

Phase 4: Agent workflow

Deliverables:

- `CLAUDE.md`.
- `AGENTS.md`.
- Reading profile.
- Tag registry.
- AI review template.
- AI review validation.
- Example prompts.
- Human/AI ownership checks.

Completion criteria:

- Claude Code produces a valid review.

- Codex produces a valid review.
- Neither agent changes human-owned fields.
- Reviews remain compact.
- Review scope and provenance are recorded.

Phase 5: Tag synchronization

Deliverables:

- Local-write authorization.
- Credential storage.
- Tag reconciliation.
- Dry-run output.
- Add-only tag patches.
- Version-conflict handling.
- Optional Web API fallback.

Commands:

```
research authorize
research tags <citekey>
research push-tags <citekey> --dry-run
research push-tags <citekey>
```

Completion criteria:

- Approved tags appear in Zotero.
- Existing Zotero tags remain unchanged.
- Suggested tags are not pushed.
- Conflicting writes do not overwrite newer Zotero changes.

Phase 6: Obsidian dashboards and polish

Deliverables:

- Inbox Base.
- Reading Queue Base.
- AI Reviewed Base.
- Recommended Reading Base.
- Human Read Base.
- Documentation.
- Troubleshooting guide.
- End-to-end acceptance test.

Completion criteria:

- The literature workflow is usable entirely through Zotero, Obsidian, and the CLI.
- The system can be installed from a clean checkout.
- The README contains a complete setup guide.

30. Recommended implementation order

The practical implementation sequence should be:

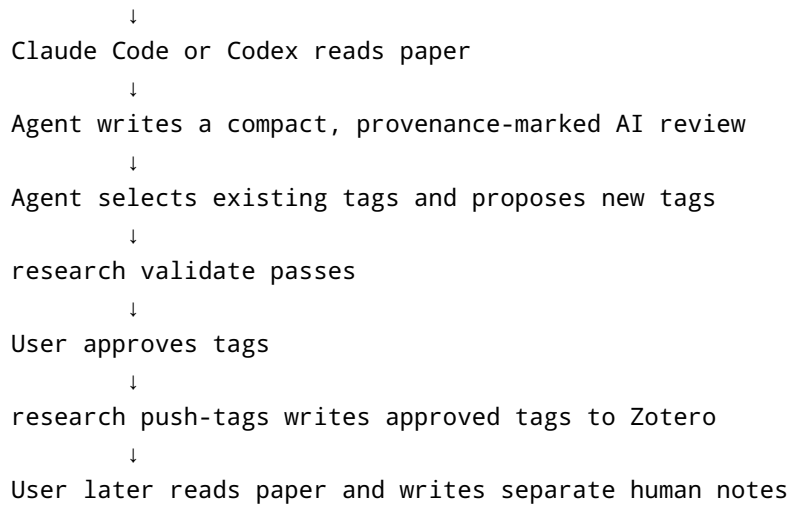
1. Create the repository and CLI skeleton.
2. Implement `research doctor`.
3. Implement Zotero item reads.
4. Implement Better BibTeX citation-key lookup.
5. Implement paper-note creation.
6. Implement safe frontmatter updates.
7. Implement incremental sync.
8. Implement PDF attachment resolution.
9. Implement PDF extraction.
10. Write agent instructions and templates.
11. Implement validation.
12. Test Claude Code review workflow.
13. Test Codex review workflow.
14. Implement tag dry runs.
15. Implement Zotero write authorization.
16. Implement tag pushes.
17. Add Obsidian dashboards.
18. Complete integration testing and documentation.

Tag writes should come after read-only synchronization and Markdown safety are stable.

31. Definition of done for version 0.1

Version 0.1 is complete when the following workflow is reliable:

```
Add paper to Zotero
  ↓
Run research sync
  ↓
Paper note appears in Obsidian
  ↓
Run research review-context <citekey>
```



The final system must guarantee:

- Zotero remains intact.
- Human notes remain intact.
- AI notes remain clearly labelled.
- AI review never changes human reading status.
- Tag writes are explicit and add-only.
- Every AI recommendation states its source coverage.
- Every generated cache can be rebuilt.
- Both Claude Code and Codex can follow the same workflow.
- The repository remains ready for later RAG and knowledge-base extensions.

32. Future extension points

The following additions should be possible without changing the paper-note identity or human/AI separation model:

- SQLite FTS5 indexing.
- Embedding-based semantic retrieval.
- MCP server exposing paper search and retrieval.
- Research-project links.
- Concept notes.
- Claim and evidence notes.
- Cross-paper synthesis.
- Automated literature reviews.
- Contradiction detection.
- Citation-aware manuscript drafting.
- Experiment and dataset notes.
- Scheduled Zotero synchronization.
- Automatic review queues.

- AI recommendation calibration against the user's actual ratings.

The Zotero item key, citation key, structured frontmatter, concise AI review, controlled tags, and provenance metadata form the stable foundation for these later stages.